LAPORAN UJIAN TENGAH SEMESTER

Mata Kuliah: Pembangunan Perangkat Lunak (Kelas A)

Nama : Muhammad Fahmi Idrus

NIM : 2310511079

Studi Kasus : Sistem Manajemen Kos (GitHub, Google dan Facebook OAuth)

Link Repository: https://github.com/fhmi18/uts-pplos-a-2310511079

Link Video Demo: https: //youtu.be/_WtI4UGb0oo

A. Pendahuluan

Sistem Manajemen Kos ini adalah sebuah platform yang menjembatani antara pemilik kos (Owner) dan penyewa kos (Tenant). Sistem ini melingkupi fitur listing properti/kamar, manajemen fasilitas, proses booking, hingga pencatatan riwayat pembayaran sewa. Sistem ini dibangun sepenuhnya menggunakan arsitektur Microservices dan API Gateway sebagai pintu masuk tunggal (Single Entry Point).

B. Arsitektur Sistem dan Justifikasi Pemisahan Service

Sistem ini memecah fitur-fitur utamanya menjadi 3 Microservice Independen dan 1 API Gateway:

- API Gateway (Node.js): Bertugas sebagai router, rate limiter, dan lapis keamanan utama (JWT Middleware).
- Auth Service (Node.js): Mengurus registrasi, login lokal, login OAuth (GitHub), dan token JWT.
- Property Service (PHP/CodeIgniter 4): Mengurus manajemen properti, kamar, dan fasilitas.
- Booking & Payment Service (Node.js): Mengurus proses pesanan kos dan pencatatan pembayaran.

Pemecahan fitur ke dalam beberapa service independen di atas dilakukan berdasarkan pertimbangan berikut jika dibandingkan dengan arsitektur Monolitik:

- Isolasi Kegagalan (Fault Tolerance)
  Pada aplikasi monolitik, jika terjadi kerusakan (misal error pada logika kalkulasi pembayaran di Booking Service), seluruh aplikasi termasuk fitur pencarian kamar akan ikut mati. Dengan microservices, down-nya Booking Service tidak akan mempengaruhi pengguna yang sedang mencari kamar di Property Service.
- Skalabilitas yang Tepat Sasaran
  Modul pencarian kamar (Property) biasanya memiliki lalu lintas (traffic) baca yang jauh lebih tinggi daripada modul pembayaran. Dengan microservices, kita dapat menaikkan kapasitas (scale-up) server khusus untuk Property Service saja, sehingga jauh lebih hemat biaya.

- Kebebasan Teknologi (Polyglot)
  Arsitektur ini memungkinkan penggunaan teknologi yang paling optimal untuk tugas tertentu. Fitur Property dibangun menggunakan PHP CodeIgniter 4 karena keandalannya dalam pola MVC dan manajemen relasi database. Sementara fitur Gateway, Auth, dan Booking menggunakan Node.js (Express) karena sifatnya yang asynchronous dan sangat cepat menangani I/O network serta routing.

C. Isolasi Database dan Relasi (REST API + Database)

Sesuai dengan prinsip microservices, setiap service memiliki skema database tersendiri yang terisolasi dan tidak diperkenankan saling melakukan JOIN tabel secara langsung.

- DB Auth: Menyimpan tabel users dan refresh_tokens.
- DB Property (Relasional): Menggunakan MySQL/MariaDB dengan 4 tabel saling berelasi, yaitu: properties, rooms, facilities, dan tabel junction room_facilities.
- DB Booking: Menyimpan tabel bookings dan payments.

(Catatan: Karena database terpisah, pertukaran data antar domain dilakukan via HTTP REST API).

D. Peta Routing API Gateway

Sistem mewajibkan Client (Frontend/Postman) untuk hanya menembak request melalui API Gateway (Port 3004). Berikut adalah peta routing (path tujuan) yang telah diimplementasikan:

Path dari Klien	HTTP Method	Service Tujuan	URL Target Internal	Deskripsi Tugas
/api/auth/*	GET, POST	Auth Service	http://localhost:3001	Menangani otentikasi, OAuth, JWT, Profil
/api/property/*	GET, POST, PUT, DELETE	Property Service	http://localhost:3002	CRUD Properti, kamar, dan fasilitas
/api/booking/*	GET, POST, PUT, DELETE	Booking Service	http://localhost:3003	Pembuatan dan update status pesanan kamar
/api/payment/*	GET, POST, PUT	Booking Service	http://localhost:3003	Mencatat dan mengonfirmasi pembayaran kos
/api/gateway/health	GET	API Gateway	(Internal)	Mengecek health status dari Gateway

E. Rincian Endpoint per Layanan

1. **Auth Service (/api/auth)**
   - POST /api/auth/register ➔ Registrasi pengguna baru.
   - POST /api/auth/login ➔ Login lokal menggunakan email & password.

- POST /api/auth/refresh ➔ Memperbarui *access token* menggunakan *refresh token*.
- POST /api/auth/logout ➔ Logout (Gateway akan mem-blacklist token ini).
- GET /api/auth/github/login ➔ Login OAuth via GitHub.
- GET /api/auth/google/login ➔ Login OAuth via Google.
- GET /api/auth/facebook/login ➔ Login OAuth via Facebook.
- GET /api/auth/callback ➔ Callback endpoint untuk OAuth.

2. Property Service(/api/property)

a. Property

- GET /api/property/health ➔ Health check untuk layanan Property.
- GET /api/property ➔ Mendapatkan daftar semua properti.
- POST /api/property ➔ Menambahkan properti baru.
- GET /api/property/{id} ➔ Mendapatkan detail properti berdasarkan ID.
- PUT /api/property/{id} ➔ Memperbarui data properti berdasarkan ID.
- DELETE /api/property/{id} ➔ Menghapus properti berdasarkan ID.

b. Fasilitas Properti

- GET /api/property/{id}/facilities ➔ Mendapatkan daftar fasilitas pada suatu properti.
- POST /api/property/{id}/facilities ➔ Menambahkan fasilitas baru ke suatu properti.
- GET /api/property/{id}/facilities/{facility_id} ➔ Mendapatkan detail fasilitas tertentu.

c. Kamar (Rooms):

- GET /api/property/{id}/rooms ➔ Mendapatkan daftar kamar dalam sebuah properti.
- POST /api/property/{id}/rooms ➔ Menambahkan kamar baru ke dalam properti.
- GET /api/property/{id}/rooms/{room_id} ➔ Mendapatkan detail kamar berdasarkan ID.
- PUT /api/property/{id}/rooms/{room_id} ➔ Memperbarui data kamar.
- DELETE /api/property/{id}/rooms/{room_id} ➔ Menghapus kamar.
- Fasilitas Kamar (Room Facilities):
- GET /api/property/{id}/rooms/{room_id}/facilities ➔ Mendapatkan fasilitas spesifik untuk sebuah kamar.
- POST /api/property/{id}/rooms/{room_id}/facilities ➔ Menambahkan fasilitas ke dalam kamar.

3. Booking Service (/api/booking)

a. Booking

- GET /api/booking ➔ Mengambil semua data pesanan (Logika berbeda antara tenant/owner).

- POST /api/booking ➔ Membuat pesanan baru (Hanya untuk role tenant).
- GET /api/booking/{id} ➔ Mengambil detail pesanan berdasarkan ID.
- PUT /api/booking/{id} ➔ Memperbarui status atau catatan pesanan.
- DELETE /api/booking/{id} ➔ Membatalkan pesanan.
- GET /api/booking/availability/{roomId} ➔ Mengecek ketersediaan kamar.

b. Payment

- POST /api/payment/booking/:bookingId ➔ Mencatat pembayaran baru untuk sebuah pesanan.
- GET /api/payment/:paymentId ➔ Mengambil detail spesifik dari sebuah pembayaran.
- PUT /api/payment/:paymentId/status ➔ Memperbarui status pembayaran (misalnya dari pending menjadi success).

F. Komunikasi Inter-Service

Service PHP (Property) tidak hanya berdiri sendiri, melainkan di-konsumsi secara aktif oleh service lain. Implementasi dalam sistem ini adalah ketika pengguna melakukan booking kamar di Booking Service (Node.js), service tersebut menggunakan Axios (HTTP Client) untuk menembak endpoint GET /api/property/{id}/rooms/{id} ke Property Service (PHP) guna memvalidasi eksistensi dan detail harga kamar.

G. Otentikasi (JWT) dan Otorisasi (OAuth)

1. Manajemen JWT dan API Gateway Middleware

a. Sistem menghasilkan Access Token yang berumur pendek (≤ 15 menit) dan Refresh Token berumur panjang (≤ 7 hari).

b. Global Middleware: Pengecekan JWT dilakukan sepenuhnya secara terpusat (middleware) di lapisan API Gateway, bukan di-hardcode secara manual di tiap controller backend.

c. Logout & Blacklist: Saat user melakukan logout, JWT tersebut dimasukkan ke memori Blacklist di Gateway sehingga token seketika menjadi tidak valid (invalidation).

d. Rate Limiting: Gateway membatasi lalu lintas maksimal 60 request/menit, dan batas yang lebih ketat (10 request/15 menit) khusus untuk endpoint /login guna mencegah serangan brute-force.

2. Otorisasi OAuth

a. Sistem menerapkan login alternatif menggunakan GitHub OAuth dengan metode Authorization Code Flow (menukarkan code dari callback menjadi access token di backend).

b. Profil dari GitHub, Google dan Facebook (Email, Nama, Foto Profil/Avatar) secara otomatis dipetakan ke dalam tabel users lokal dengan flag oauth_provider.

H. Penutup

Keseluruhan sistem telah mengimplementasikan RESTful API standar dengan penggunaan semantik HTTP Method (GET, POST, PUT, DELETE) yang benar, validasi input yang benar serta penerapan filter dan pagination (page, per_page) pada endpoint listing. Arsitektur yang dibangun telah sepenuhnya mematuhi standar Service-Oriented yang diwajibkan dalam rubrik penilaian.